

Ejercicio 17

¿Cuál es el orden de ejecución del siguiente fragmento de código? Calcule la función de tiempo de ejecución.

```
for (int i = 0; i < n*n ; i ++)  
    for (int j = i % n; j >= 0; j--)  
        System.out.println("j:"+j);
```

Como el primer for va desde 0 hasta $i < n^2$ y el índice aumenta de a uno, la iteración se ejecuta $n^2 - 1$ veces. Por otro lado, en la sumatoria interna, al ser el primer valor del índice el resto de la división $\frac{i}{n}$. Si, por ejemplo, ponemos valores a n :

- Con $n = 2$
 1. Primera iteración
 - (a) $j = 1\%2 = 1$
 - (b) $j = 0$Se ejecuta 2 veces
 2. Segunda iteración
 - (a) $j = 2\%2 = 0$Se ejecuta una vez
 3. Tercera iteración
 - (a) $j = 3\%2 = 1$
 - (b) $j = 0$Se ejecuta 2 veces
 4. cuarta iteración
 - (a) $j = 4\%2 = 0$Se ejecuta una vez
- Con $n = 3$
 1. Primera iteración
 - (a) $j = 1\%3 = 1$
 - (b) $j = 0$

- Se ejecuta 2 veces
2. Segunda iteración
 - (a) $j = 2\%3 = 2$
 - (b) $j = 4\%3 = 1$
 - (c) $j = 0$

Se ejecuta 3 veces
 3. Tercera iteración
 - (a) $j = 3\%3 = 0$

Se ejecuta 1 vez
 4. Cuarta iteración
 - (a) $j = 4\%3 = 1$
 - (b) $j = 0$

Se ejecuta 2 veces
 5. Quinta iteración
 - (a) $j = 5\%3 = 2$
 - (b) $j = 1$
 - (c) $j = 0$

Se ejecuta 3 veces

Esta me re costó jajaja.

El segundo for se ejecuta $i\%n + 1$ veces (el +1 es porque incluye el 0).

Entonces,

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{n^2-1} \left[\sum_{j=1}^{i\%n+1} c_1 \right] \\
 &= \sum_{i=0}^{n^2-1} [c_1(i\%n + 1 - 1 + 1)] \\
 &= c_1 \sum_{i=0}^{n^2-1} (i\%n + 1)
 \end{aligned}$$

Como $i\%n$ genera un múltiples veces un patrón de 0, 1, 2, ..., n-1 repetido n veces, $\sum_{i=0}^{n^2-1} (i\%n + 1) = n \sum_{k=0}^{n-1} (k + 1)$.

Por ejemplo, en el caso de $n = 3$, tenemos 2 ejecuciones (de 0 a 1) en la primera

iteración, 3 en la segunda (de 0 a 2) y 1 en la tercera (resto=0). Entonces,

$$\begin{aligned}
&= c_1 n \sum_{k=0}^{n-1} (k+1) \\
&= c_1 n \left[\sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} 1 \right] \\
&= c_1 n \left[\sum_{k=0}^0 k + \sum_{k=1}^{n-1} k + n \right] \\
&= c_1 n^2 + c_1 n \left[\frac{(n-1)n}{2} \right] \\
&= c_1 n^2 + \frac{c_1}{2} n(n^2 - n) \\
&\text{Renombrando } c_2 = \frac{c_1}{2} \text{ y } c_3 = -c_1 \\
&= c_1 n^2 + c_2 n^3 + c_3
\end{aligned}$$

Entonces, la función de tiempo de ejecución del método es

$$T(n) = c_1 n^2 + c_2 n^3 + c_3$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_1 n^2 + c_2 n^3 + c_3 \leq O(n^3)$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_1 n^2 + c_2 n^3 + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

Se tiene que demostrar que

$$T(n) = c_2 n^3 + c_1 n^2 + c_3 \leq O(n^3)$$

Para esto, tenemos que encontrar una constante $k > 0$ y un n_0 tales que

$$c_2 n^3 + c_1 n^2 + c_3 \leq k n^3 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: $c_3 2^3$
Sabemos que $n^3 \leq n^3$ Si multiplicamos a ambos lados por c_2 ,

$$c_2 n^3 \leq c_2 n^3$$

Encontramos un valor $k_1 = c_2$. Ahora, si probamos con $n_0 = 0$.

$$c_2 \cdot 0^3 \leq c_2 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_2$ y $n_0 = 0$, para $n_0 \geq 0$.

- Segundo término: $c_1 n^2$
Sabemos que $n^2 \leq n^3$ Si multiplicamos a ambos lados por c_1 ,

$$c_1 n^2 \leq c_1 n^3$$

Encontramos un valor $k_2 = c_1$. Ahora, si probamos con $n_0 = 0$.

$$c_1 \cdot 0^2 \leq c_1 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_1$ y $n_0 = 0$, para $n_0 \geq 0$.

- Tercer término: c_3
Sabemos que $1 \leq n^3$ Si multiplicamos a ambos lados por c_3 ,

$$c_3 n \leq c_3 n^3$$

Encontramos un valor $k_3 = c_3$. Ahora, si probamos con $n_0 = 1$.

$$c_3 \leq c_3 \cdot 1^3$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_3$ y $n_0 = 1$, para $n_0 \geq 1$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_2 n^3 + c_1 n^2 + c_3 \leq k_1 n^3 + k_2 n^3 + k_3 n^3$$

$$T(n) \leq (k_1 + k_2 + k_3) n^3$$

$$T(n) \leq (c_1 + c_2 + c_3) n^3$$

Renombrando $c = c_1 + c_2 + c_3$

$$T(n) \leq c n^3$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^3), \text{ con } c = c_1 + c_2 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$